

Automated Machine Learning

This presentation highlights the process of developing and deploying an AutoML application on Google Cloud Platform (GCP). The application empowers users to upload their CSV datasets, choose target variables, eliminate unnecessary features, and automatically train either classification or regression models. The entire process concludes with the generation of a detailed report, offering insights into the model's performance and usability.

Project Overview



Objective

Streamline machine learning processes for users.



Target Users

Designed for data analysts, business users, and researchers.



Technological Stack

Utilizes Python, Scikit-Learn, Flask, Docker, Git-GitHub & Google Cloud Platform.

Technology Stack



Python

The whole pipeline is created using Object Oriented Programming in Python.



Scikit-learn

Scikit-Learn is used in Feature Engineering, Training and Tuning models.



Docker

Docker used for Containerization for Deployment.



Flask

Lightweight framework for building web applications.



Google Cloud Platform

Comprehensive cloud services for deployment.

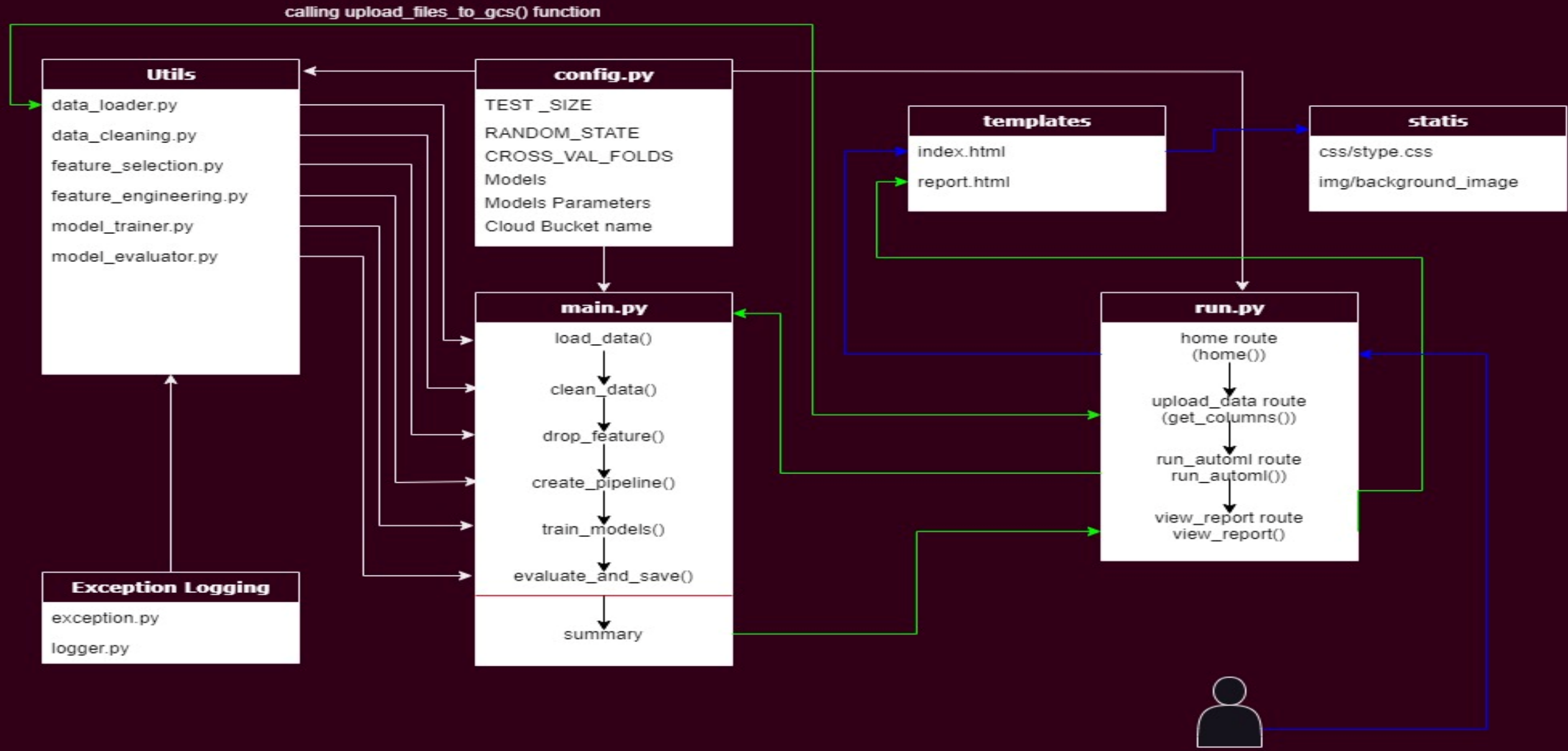


Git-Github

Version control and CI/CD for collaboration.

App Workflow Diagram

AutoML App



Features of the App

This application offers a variety of features designed to streamline data analysis and model training. Users can easily manage their datasets and receive valuable insights through automated processes.

CSV Upload

Users can easily upload their datasets in CSV format, which is a standard data format widely used in data analysis and machine learning.

Reporting

At the end of the process, the app generates a comprehensive report summarizing model performance metrics and insights, aiding users in understanding their results.



Target Variable Selection

The application allows users to specify the target variable they wish to predict, whether it's for classification or regression tasks.

Feature Management

Users can select features to drop from the dataset, enabling them to refine their models and focus on the most relevant data.

Automated Training

The app automatically detects and trains either a classification or regression model, reducing the time and effort required for model building.

Technical Implementation Overview

User Interface

CSV Upload Page: Select Target Column → Drop Features.

Training Pipeline Trigger: `/run_automl` route.

Report Page: Displays evaluation metrics, confusion matrix, CV scores.

Reports: HTML/PDF generated for download.

Data Preprocessing

Data Cleaning: Missing values imputed (median for numeric, mode for categorical).

Feature Selection: User-specified features dropped before training.

Feature Engineering: Numeric scaling, categorical encoding via `ColumnTransformer`.

Automatic Encoding: Non-numeric target labels encoded via `LabelEncoder`.

Scalability

Stateless Flask API deployed on Cloud Run (auto-scales with traffic).

Containerization: Entire pipeline packaged in Docker for portability.

Cross-validation folds configurable (`config.py`).

Model Selection & Training

Automatic Task Detection:

- Classification vs Regression decided using `ModelTrainer._determine_task_type()`.

Model Library: Logistic Regression, Decision Tree, Random Forest, SVM, KNN, XGBoost.

Hyperparameter Tuning: `GridSearchCV` integrated with parameter grids from `config.py`.

Model Persistence: Trained models uploaded back to GCS.

GCP Services

Cloud Run: Hosts the Flask web application.

Cloud Storage (GCS): Stores uploaded CSV datasets and trained models.

Cloud Build & Artifact Registry: Used for Docker image builds & deployments.

IAM Roles: Configured for secure dataset/model access.

Challenges Faced



Data Quality

Ensuring high-quality data input from users posed challenges, as varied data formats and missing values can significantly impact model performance. Therefore, pipeline is built such a way that it will handle the missing and duplicate entries so that the model performance is not compromised.



User Experience

Designing an intuitive interface required multiple iterations to accommodate users with varying levels of technical expertise. User interface is designed such a way that user need to provide minimal input to get the best performant model.



Model Interpretability

Providing users with understandable insights from the generated models was crucial, necessitating the development of clear reporting mechanisms.



Deployment Issues

Initial deployment faced challenges related to configuring GCP services correctly, requiring troubleshooting and adjustments to ensure smooth operation.

Future Enhancements

Expanded Model Types

Future iterations will include additional model types and algorithms, allowing for more complex analyses and predictions tailored to user needs.

Collaboration Features

Future versions may include collaboration tools, enabling teams to work together on datasets and model development.



Enhanced Visualization

Incorporating advanced data visualization tools will help users better interpret their datasets and model outcomes.

Real-time Predictions

Adding functionality for real-time predictions will enable users to apply their models to new data as it becomes available.

User Feedback Loop

Implementing a feedback mechanism will allow users to provide input on the app's functionality, driving continuous improvement and feature updates.

Conclusion

Impact on Users

The AutoML app significantly reduces the barrier to entry for machine learning, enabling users to derive insights from data without needing extensive technical knowledge.

Efficiency Gains

By automating the model training process, the app saves users time and effort, allowing them to focus on interpreting results rather than on technical implementation.

Future Potential

With planned enhancements, the app has the potential to evolve into a comprehensive analytics platform, further empowering users and enhancing decision-making processes.

Access Resources

1 GitHub Repository

GitHub

2 Try the App

Try AutoML

3 Feedback Channel

Contact Me

